Failure Synthesis System

Research papers, drone logging standards & public data, and prototyping plan | Prepared 8 Oct 2026 for mentor review

Key takeaways

- **No IEEE standard defines an onboard drone flight-log format.** IEEE 1936.1-2021 covers operation records at a framework level; IEEE 1937.3-2024 covers supervision-data transmission. The de facto log standards are autopilot formats: **MAVLink 2**, **PX4 ULog**, **ArduPilot DataFlash**, and **MCAP**.
- **Real drone data is available**: PX4 Flight Review public logs (CC-BY, bulk download), ALFA (real faults with ground-truth timing), RflyMAD (11 fault types, 5,629 cases), and a cyber-physical swarm dataset.
- **Gap:** no public dataset found contains *labelled cross-agent causal failure chains* (the Drone 298 -> Drone 474 scenario). We must generate these ourselves in multi-vehicle simulation with fault injection. This gap is also a defensible contribution.
- **Design recommendation:** use MAVLink message definitions as the payload-schema source for the deterministic decoder (event_type = msgid). This gives a real, documented schema set, and unknown-msgid handling maps directly onto our UNKNOWN_EVENT_TYPE rule.
- **Prior art validates the architecture:** per-record CRC framing (LevelDB, Kafka), segment/chunk time indexes with index recovery from truncated files (MCAP), and two-stage collect-then-reason LLM RCA (RCACopilot).

1. Research papers & technical references

Grouped by the subsystem each one informs. The "Use in FSS" column records what to take from each source, not a summary of it.

A. Crash-consistent storage (Java core)

Reference	Venue / link	Use in FSS
Pillai et al., *All File Systems Are Not Created Equal: On the Complexity of Crafting Crash-Consistent Applications*	OSDI 2014 usenix.org	Core citation for FR-07 / NFR-02. File-system persistence properties vary across file systems, so the store must not assume writes are atomic or ordered. Their ALICE tool explores crash states; replicate a lightweight version as our crash harness.
Mohan et al., *ARIES: A Transaction Recovery Method Supporting Fine-Granularity Locking and Partial Rollbacks Using Write-Ahead Logging*	ACM TODS 1992 doi:10.1145/128765.128770	Canonical write-ahead-log recovery. Our store is simpler (append-only, no undo), but we borrow the vocabulary: durability point, log sequence, redo scan.
LevelDB log format (design doc)	github.com/google/leveldb	Nearly our design: 32 KB blocks, a per-record checksum + length + type header, and recovery that skips corrupt data to the next block. Key difference: LevelDB fragments records across blocks (FIRST/MIDDLE/LAST), while we forbid it (FR-04). Cite this as a deliberate choice.
Kreps, Narkhede, Rao, *Kafka: a Distributed Messaging System for Log Processing*	NetDB 2011 paper PDF	Segmented append-only log, a CRC per message, and reliance on the OS page cache. This is the reference for segment files plus sparse offset index, and for decoupling producers from the writer (FR-02).
MCAP format specification (Foxglove)	github.com/foxglove/mcap	Robotics log container. Chunks carry start/end time and a CRC, a summary index sits at the end, and recovery can rebuild the index from a truncated file. This maps closely to our segment + block metadata + recovery model. Strongest single piece of prior art.

B. Time & causality

Reference	Venue / link	Use in FSS
Lamport, *Time, Clocks, and the Ordering of Events in a Distributed System*	CACM 1978 doi:10.1145/359545.359563	Formal justification for handoff rule 19: producer timestamps from unsynchronised clocks cannot establish causal order. Cite it wherever the synthesis reports clock uncertainty.

C. UAV fault data & drone forensics

Reference	Venue / link	Use in FSS
Keipour, Mousaei, Scherer, *ALFA: A Dataset for UAV Fault and Anomaly Detection*	IJRR 2021 arXiv:1907.06268	47 processed autonomous fixed-wing flights with engine and control-surface faults, each with ground-truth fault time and type. Use as the real-flight evaluation set for single-agent synthesis.
Le, Jin, Cui, Dai, Quan, *RflyMAD: A Dataset for Multicopter Fault Detection and Health Assessment*	IJRR 2025 arXiv:2311.11340	114 GB, 11 fault types, 5,629 cases (SIL, HIL and real flight). Largest labelled multicopter fault set. Use a subset for decoder and synthesis testing.
Clark, Meffert, Baggili, Breitinger, *DROP (DRone Open source Parser) your drone: Forensic analysis of the DJI Phantom III*	Digital Investigation (DFRWS) 2017	Forensic parsing of encrypted, proprietary flight logs. Motivates our rule that uninterpretable bytes stay preserved and are reported, not guessed.
Who Watches the New Watchmen? The Challenges for Drone Digital Forensics Investigations (authors: verify)	arXiv:2105.10917	Surveys drone forensics. Notes that methods tied to one vendor's proprietary logs do not generalise, and describes MAVLink-based extraction. Supports our configurable-envelope design (NFR-08).
DroPTC: sentence-level drone flight log forensics using contrastive learning and explainable AI	uni-augsburg OPUS	Closest related work on the explanation side: localising problems inside flight-log messages for root-cause attribution. Position FSS against it: they classify text logs, while we reconstruct cross-agent binary evidence.

D. LLM-based log analysis & root-cause analysis (Python layer)

Reference	Venue / link	Use in FSS
Chen et al., *Automatic Root Cause Analysis via Large Language Models for Cloud Incidents* (RCACopilot)	EuroSys 2024 arXiv:2305.15778	Two stages: deterministic diagnostic collection, then LLM root-cause prediction with an explanation. Same split as our Java retrieval -> Python synthesis. Reports RCA accuracy up to 0.766 on a year of Microsoft incidents. Main justification for the architecture.
Liu et al., *Interpretable Online Log Analysis Using LLMs with Prompt Strategies* (LogPrompt)	ICPC 2024 arXiv:2308.07610	Zero-shot, interpretable log analysis through prompt strategies. Use their prompt-strategy comparisons when designing the synthesis prompt.
Qi et al., *LogGPT: Exploring ChatGPT for Log-Based Anomaly Detection*	2023 arXiv:2309.01189	Baseline for LLM log reasoning on text logs. Contrast with FSS, where the LLM never sees raw bytes.
Lewis et al., *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*	NeurIPS 2020 arXiv:2005.11401	Foundational RAG citation for the domain-knowledge layer.

Note: ARIES, Lamport and RAG are cited from standard references, and one author list is marked (verify). Check them on the publisher page before they go into the report bibliography.

2. Drone logging standards & availability of real data

2.1 Formal standards (IEEE / ASTM)

Standard	What it covers	Relevance to FSS
IEEE 1936.1-2021 Drone Applications Framework	Framework for drone applications: application classes, facility requirements (flight control, GCS, data link), safety and management. The operations section specifies operation records and reports, including data classification, collection/processing, record/analysis, and a data reference format. 28 pp.	The **only IEEE text that touches logging**, at the level of operation records rather than a binary telemetry format. Get it through PEC's IEEE Xplore access and reuse its data classification for our event_type taxonomy.
IEEE 1937.3-2024 Flight Data Transmission via Short Message	Content, protocol and transmission method for UAV flight *supervision* data over GNSS (BeiDou) short messages. 21 pp.	Defines a minimal supervision-data field set (position, state). Usable as a reference schema for a POSITION event. It is a transport standard, not a log store.
IEEE P1937.13 / P1937.16	Autonomy-level taxonomy; civil UAS cybersecurity framework. Both in development.	Peripheral. P1937.16 is relevant only if we add tamper-evidence (signed records).
ASTM F3411-22a Remote ID and Tracking	Message formats, transmission methods and performance for broadcast and network Remote ID (FAA Part 89 means of compliance).	Small, fully specified binary messages with open-source encoders and decoders. Good as **decoder test vectors**.

Conclusion: the IEEE drone standards cover application frameworks, payload interfaces, airspace and transmission. None specifies an onboard event-log format or crash-recovery semantics. That gap supports the project's motivation.

2.2 De facto logging formats used on real drones

Format	Structure (what matters for us)	How FSS uses it
MAVLink 2 mavlink.io	Binary telemetry protocol. Header carries seq, sysid (vehicle), compid, a 24-bit msgid and payload length. Messages are defined in XML. On the wire, fields are reordered largest-first. A CRC-16/MCRF4XX is seeded with a per-message CRC_EXTRA derived from the definition, so a receiver can detect a schema mismatch. Max packet 280 bytes.	**Recommended payload-schema source.** sysid -> source_id, msgid -> event_type, XML -> decoder schema. CRC_EXTRA is a ready-made schema-version check. The Python decoder can reuse pymavlink definitions.
PX4 ULog docs.px4.io	Self-describing log of uORB topics: a 16-byte header, then definitions ('F' format messages), then data. Every subscribed format includes a timestamp field.	Source of realistic payloads. A ULog replayer converts topics to EventRecords and streams them into the Java ingest (parse with pyulog).
ArduPilot DataFlash (.bin)	Self-describing: FMT records declare type id, length, name, format string and column labels. Messages start with header bytes 0xA3 0x95. Parsers can resync after corruption by scanning for the next valid header.	Second replay source. Its resync-on-corruption behaviour is a reference for our recovery scan.
MCAP MCAP spec	Container for pub/sub messages: schemas and channels embedded; chunks with message start/end time and CRC; a summary index at the end; nanosecond timestamps.	Design reference for segment/block metadata, and an optional export format so logs open in Foxglove.

2.3 Is such data available on the internet? Yes, with one gap

Dataset	Content	Multi-agent?	Fault labels?	Use in FSS
PX4 Flight Review public logs logs.px4.io	Large public ULog corpus across vehicle types, boards, PX4 versions and flight modes. CC-BY. Bulk download via download_logs.py, with filters by mode or airframe.	No (one vehicle per log)	No	Realistic event streams, ingest load tests, decoder coverage.
ALFA (CMU AirLab) doi:10.1184/R1/12707963	Real fixed-wing flights: 47 processed, 23 engine-failure and 24 control-surface-fault scenarios, plus raw data. Loader tools in C++, MATLAB and Python.	No	Yes (time + type)	Real-flight evaluation of single-agent synthesis.

Dataset	Content	Multi-agent?	Fault labels?	Use in FSS
RflyMAD (Beihang) rfly-openha	Multicopter, 11 fault types x 6 flight states; 2,566 SIL + 2,566 HIL + 497 real cases; 114 GB.	No	Yes	Fault-type coverage; download a subset only.
Cyber-Physical Dataset of Drone Swarms in Coordinated Formations IEEE DataPort	Swarm in formations: network telemetry plus UAV position, timestamp-aligned, CSV; simulated and real.	**Yes**	Cyber-attack labels	Closest public data to our multi-source retrieval case. DataPort may need a login.
U2UData+ (AAAI 2026) arXiv:2509.00055	15-UAV collaborative flights, 120 h, LiDAR and RGB plus environmental values.	Yes	No	Perception-heavy and multi-TB. Not needed for the prototype.

Gap: no dataset found has labelled cross-agent failure chains (agent A's fault causing agent B's behaviour).
Plan: generate these in PX4 SITL multi-vehicle simulation, inject faults on one vehicle, script the reaction on a neighbour, and record the injected cause as ground truth. PX4 has a built-in failure-injection facility, but its exact commands and parameters are still to be checked in the PX4 docs.

2.4 Notes for the design

- Both PX4 and ArduPilot logs are **self-describing** (format records inside the file). Our design keeps schemas outside the store, in Python config. Be ready to explain to the mentor why: the store stays protocol-agnostic and Java never interprets payloads.
- MAVLink's **sysid** is a natural source_id, and **msgid** (24-bit) fits a u32 event_type. Use MAVLink's mapping rather than inventing one.
- ULog timestamps are in microseconds since boot, and MCAP uses nanoseconds. Fix **u64 nanoseconds** in our envelope and store the clock basis (boot vs UTC) per source in metadata.
- MAVLink CRC_EXTRA is the precedent for a **schema_version** field: add it to DecodedEvent.

3. Next steps: prototyping plan

Ordering principle: freeze the contract first, run the Python layer and the Java core in parallel against shared fixtures, and leave segment indexing until the simple store has been proven crash-safe.

3.1 Contract to freeze this week (v0)

```
Wire envelope (little-endian):
magic u16 | version u8 | flags u8 | source_id u32 | timestamp_ns u64 | event_type u32 |
payload_len u32 | payload[..]

On-disk record: record_len u32 | crc32c u32 | envelope bytes (crc covers len + envelope)
Block: fixed size (e.g. 64 KiB), zero-preallocated, single source, no record crosses a boundary

retrieve(source_id | ALL, t_start_ns, t_end_ns, event_type?, max_events) -> EventRecord[] +
truncated flag
Transport Java -> Python: gRPC or HTTP+JSON (payload base64); Python never opens store files
```

3.2 Milestones

#	Milestone	Work	Exit criterion
M0	Contract + fixtures	Freeze the envelope and retrieval API (3.1). Write a fixture generator for the 298/474 scenario with an unknown type 0x31, partially decodable, malformed and unknown-enum cases.	Golden binary fixtures plus expected DecodedEvent JSON checked into the repo.
P1	Python decoder (no Java)	MockRetriever, schema loader (YAML), primitive decoders with bounds checks, enum mapping, decode_status paths.	Every handoff rule (sections 10-14) has a unit test; 100% of fixtures match expected output.
P2	Real-data replayer	Pull 20-50 logs with download_logs.py; convert ULog topics to EventRecords using pyulog; map 5-8 event types to schemas (or MAVLink defs).	Replayer streams real events at 1x or Nx rate to a socket or file.
J1	Java single-writer store	Append-only file, CRC framing, zero-prealloc blocks, recovery = scan until len==0 or CRC mismatch, then truncate. Full-scan retrieve.	Crash harness passes: kill -9 under load, truncation at every offset, random bit flips. Recovered set is always a valid prefix.
J2	Concurrent ingest	TCP server, N producer connections, bounded queue, single writer thread, group commit (force every N ms or N events), backpressure.	Throughput (events/s) reported for force=never / group / every-write; zero loss of acknowledged events on process kill.
J3	Segments + block metadata	Time segments (by receive time), per-source block lists, min/max ts per block, sealed vs active segment, metadata rebuilt from data on recovery.	JMH benchmark: bounded-window query latency stays flat as total history grows (vs J1 full scan).
S1	Multi-agent scenarios	PX4 SITL multi-vehicle + fault injection; 5-10 scripted cross-agent chains with injected ground truth; vary clock skew.	Labelled scenario set with known cause, agents and time windows.
I1	Integration + RAG + LLM	Swap MockRetriever for the Java client; domain docs (MAVLink enum docs, PX4 docs) into RAG; FailureSynthesis JSON; post-LLM validator.	End-to-end answer to 'why did 474 leave formation' on an S1 scenario.
E1	Evaluation + dashboard	Metrics below; minimal dashboard (query form, timeline, synthesis, data-gaps panel).	Results table for the report.

3.3 Evaluation metrics (decide now, so the prototype logs them)

Area	Metric
Durability	Acknowledged-event loss after kill -9 (target 0); recovery time vs active-segment size; % of crash states with a valid-prefix outcome.
Performance	Ingest events/s vs number of producers and fsync policy; p50/p99 retrieve latency for 10 s / 60 s windows at 1 GB / 10 GB history.

Area	Metric
Decoder	Fixture pass rate; never emits a value for an out-of-bounds field (malformed -> MALFORMED_PAYLOAD).
Synthesis	Root-cause attribution accuracy on S1 / ALFA scenarios; citation validity (every contributing event exists in the evidence set); unknown-preservation (no interpretation of opaque bytes or unmapped enums); clock-uncertainty stated when cross-agent ordering is used.

3.4 Risks

Risk	Mitigation
Java store slips; Python is blocked.	Python runs entirely on MockRetriever + fixtures until I1; the contract is frozen at M0.
Durability scope is ambiguous (process crash vs power loss).	Decide with the mentor. Default: process crash guaranteed; OS crash / power loss best-effort via group-commit force().
Segmenting by producer time misses skewed events.	Segment by receive time; keep the producer timestamp as evidence; pad query windows by a skew tolerance.
No public multi-agent failure data.	S1 synthetic scenarios with ground truth; ALFA for real single-agent faults.
LLM invents causes or citations.	Structured output + validator; unknowns passed as explicit gaps; temperature 0; human-readable evidence list.

3.5 For tomorrow's mentor meeting

- **Show:** the SRS / DFD / class / sequence set, the Python handoff, and this document (standards gap, data sources, plan).
- **Demo if ready:** fixture -> decoder -> DecodedEvent JSON, with UNKNOWN_EVENT_TYPE, PARTIALLY_DECODED and MALFORMED_PAYLOAD outputs.
- **Ask the mentor to decide:** (1) durability scope; (2) adopt MAVLink as the payload schema source, yes or no; (3) segment clock basis (receive vs producer time); (4) per-source blocks vs single log + index; (5) whether a synthetic multi-agent scenario set is acceptable as the evaluation basis.
- **Fix in the docs:** SRS 5.2 should use min/max timestamp, not first/last (ordering is not guaranteed); add the one-source-per-block rule to the SRS; define the timestamp unit (u64 ns) and the segment clock basis.